Student 13 **Jayaraman Renuka Devi** 192573007RD 5 / 5 submitted

Q1. ATM Withdrawal – Exception Handling Debugging

CODE

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            System.out.println("Enter account balance: ");
            int balance=sc.nextInt();
            System.out.println("Enter withdrawal amount: ");
            int amount=sc.nextInt();
            if(amount<=0){
                System.out.println("Invalid withdrawal amount");
            }
            else if(amount>balance){
                System.out.println("Insufficient balance");
            }
            else{
                int remaining=balance-amount;
                System.out.println("Withdrawal successful");
                System.out.println("Remaining balance= " + remaining);
            }
        }
        catch(InputMismatchException e){
            System.out.println("Invalid input");
        }
        catch(ArithmeticException e){
            System.out.println("Transaction completed");
        }
        catch(Exception e){
            System.out.println("Unexpected error");
        }
        finally{
            System.out.println("Transaction completed");
        }
    }
}
```

INPUT

```
10000
-500
```

OUTPUT

```
Enter account balance:
Enter withdrawal amount:
Invalid withdrawal amount
Transaction completed
```

Q2. Hospital Registration – NullPointerException Debugging

CODE

```
import java.util.Scanner;
//import java.util.InputMismatchExcpetion;
public class main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            String inputname=sc.nextLine();
            String name;
            if(inputname.equals("NULL")){
                name=null;
            }
            else{
                name=inputname;
            }
            int age=sc.nextInt();
            if(name==null){
                System.out.println("Patient name is unavailable");
            }
            else{
                System.out.println("Patient Name: " + name.toUpperCase());
            }
            if(age>=18){
                System.out.println("Adult patient");

            }
            else{
                System.out.println("Minor Patient");
            }
        }
        catch(java.util.InputMismatchException e){
            System.out.println("Invalid age input");
        }
        catch(Exception e){
            System.out.println("Unexpected error");
        }
        finally{
            System.out.println("Registration completed");
        }
    }
}
```

INPUT

```
meena
absd
```

OUTPUT

```
Invalid age input
Registration completed
```

Q3. E-Commerce Product Search – Array Exception Debugging

CODE

```
import java.util.Scanner;
public class main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            String[] products={"Laptop","Mobile","Tablet","Headphone","Keyboard"};
            int productnumber=sc.nextInt();
            if(productnumber<1 || productnumber>5){
                System.out.println("Invalid product number");
            }
            else{
                System.out.println("Selected product: " + products[productnumber-1]);
            }
        }
        catch(java.util.InputMismatchException e){
            System.out.println("Invalid input");
        }
        finally{
            System.out.println("Search completed");
        }
    }
}
```

INPUT

```
6
```

OUTPUT

```
Invalid product number
Search completed
```

Q4. Railway Reservation – Synchronization Debugging

CODE

```
public class main{
    static class Reservation{
        int seats=1;
        synchronized void bookseat(String customer){
            if(seats>0){
                System.out.println(customer + " is booking a seat");
                try{
                    Thread.sleep(1000);
                }
                catch(InterruptedException e){
                    System.out.println("Thread interrupted");
                }
                seats--;
                System.out.println(customer + " booking successfull");
            }
            else{
                System.out.println(customer + " - No seta  available");
            }
        }
    }
    static class Customer extends Thread{
        Reservation reservation;
        String customername;
        Customer(Reservation reservation, String customername){
            this.reservation=reservation;
            this.customername=customername;
        }
        public void run(){
            reservation.bookseat(customername);
        }
    }
    public static void main(String[] args){
        Reservation reservation=new Reservation();
        Customer c1=new Customer(reservation,"Customer 1");
        Customer c2=new Customer(reservation, "Customer 2");
        c1.start();
        c2.start();

    }
}
```

OUTPUT

```
Customer 1 is booking a seat
Customer 1 booking successfull
Customer 2 - No seta  available
```

Q5. Bank Account – Race Condition Debugging

CODE

```
public class main{
    static class bankacc{
        int bal=10000;
        synchronized void withdraw(int amt){
            if(bal>=amt){
                System.out.println(Thread.currentThread().getName() + "withdrawing" + amt);
                try{
                  Thread.sleep(100);
                }
                catch(InterruptedException e){
                   System.out.println("Interrupted");
                }
                bal=bal-amt;
                System.out.println(Thread.currentThread().getName() + "withdrawal successfull");
             }
            else{
                System.out.println(Thread.currentThread().getName() + "Insufficient balance");
           }
        }
    }
    static class ATM extends Thread{
        bankacc acc;
        int amt;
        ATM(bankacc acc,int amt){
            this.acc=acc;
            this.amt=amt;

        }
        public void run(){
            acc.withdraw(amt);
        }
    }
    public static void main(String[] args) throws InterruptedException{
        bankacc acc=new bankacc();
        ATM atm1=new ATM(acc,7000);
        ATM atm2=new ATM(acc,6000);
        atm1.setName("ATM-1");
        atm2.setName("ATM-2");
        atm1.start();
        atm2.start();
        atm1.join();
        atm2.join();
        System.out.println("Final balance= " + acc.bal);
    }
}
```

OUTPUT

```
ATM-1withdrawing7000
ATM-1withdrawal successfull
ATM-2Insufficient balance
Final balance= 3000
```